TECHNICAL EVALUATION PROJECT

ParaChat Distributed System

A Lightweight, Multi-Threaded Socket Architecture for Concurrent Network Communication

AUTHOR
Natasha Khan

COURSE CONTEXT
Distributed Systems & Networks

DATE REFERENCE
June 13, 2026

Introduction & Problem Statement

The Paradigm

As detailed in **ParaChat_Project_Report.docx**, ParaChat is designed to handle multiple concurrent nodes across a secure TCP IP network topology.

The system leverages a client-server architecture model to isolate socket networking interfaces from high-overhead runtime system layers.

Conventional Bottlenecks

Most standard chat applications require heavy application runtimes, large database structures, and bloated interface rendering blocks.

The ParaChat Solution:
A highly asynchronous pythonic core optimized with multi-threaded loop registries to execute socket data transfers with minimal processing delays.

System Technical Objectives

Real-Time Processing

Provides zero propagation delay metrics by delivering instant packet routes through localized looping matrices.

Threaded Scaling

Offloads dynamic client pipelines to dedicated server worker sockets preventing standard block exceptions.

Session Tokens

Enforces clean string-token authentication routines during the initial handshake sequence protocols.

Unified Broadcast

Intercepts packet flows and sweeps across live socket tables to dispatch communications universally.

Technical Specifications Network Details

System Attribute	Network Parameter Target	Implementation Objective
Transport Layer Protocol	TCP/IP Protocol Stack	Guarantees reliable byte-stream network message deliveries without data dropouts.
Default Target Host IP	`127.0.0.1` (Localhost Loopback)	Safeguards internal cluster testing flows across local terminal hardware networks.
Configured Active Port	Port `5000`	Establishes dedicated communication tunnel clear of standard local OS interfaces.
Buffer Byte Allocation	`1024 Bytes`	Optimizes memory pipeline footprints while supporting standard text transmission formats.

Distributed System Data Flow

Sequential Communication Pipeline

The linear progression of a message payload follows a highly structured routing path to ensure instant synchronization across all active sockets:

GUI Entry → Client Socket → TCP Tunnel → Server Hub → Multicast Loop

Threads listen asynchronously, bypassing main-loop block states inside Tkinter GUI rendering blocks.

< 1ms

Propagation Delay Limit

Achieved by isolating standard I/O streams using threading libraries natively inside Python.

Server Threading Logic Architecture

```
# Dynamic client array and broadcasting matrix snippet def broadcast(message,
sender_client=None): for client in clients: if client ≠ sender_client: try:
client.send(message.encode()) except: if client in clients: clients.remove(client)
def handle_client(client): while True: try: message = client.recv(1024).decode()
if not message: break broadcast(message, client) except: break
```

Server Worker Management

The server instantiates a central socket handler mapping unique consumer connection blocks into parallel system lists:

→ • **Asynchronous Routing:** Workers route independent data flows safely.

→ • **Exception Routing:** Broken connections are cleaned dynamically from state registers.

Asynchronous Stability Note: Dynamic disconnection cleanup processes discard thread resources instantly upon interface exits.

UI Mechanics & Theme Styling

Asynchronous Tkinter Design

The UI relies on a sleek, dark-theme layout using strict #1e1e1e blocks designed for low visual strain during high operational loads.

Dynamic Console Color Codes:

- **Cyan Highlights:** Identifies self-sent message packages inside standard scrolling blocks.
- **White Blocks:** Renders active messages arriving from remote client systems.
- **Light Green Notes:** Displays dynamic server notifications and state logs.

Validation & Test Ledger Strategy

- **TC-001 (Server Initializing):** Socket registers successfully and starts listening on port 5000. **[PASSED]**
- **TC-002 (Concurrency Baseline):** Spawns 3 distinct client execution pipelines concurrently without locking system threads. **[PASSED]**
- **TC-003 (Broadcast Alignment):** Relays client user data packages universally, showing near-zero propagation delay. **[PASSED]**
- **TC-004 (Exception Safe Guard):** Closing the client interface forces socket destruction and system session registry cleans. **[PASSED]**

System Execution Output Logs

Active Server Log Pipeline

```
🟢 ParaChat Server running on 127.0.0.1:5000
Waiting for connections ...
🟡 New connection from: ('127.0.0.1', 54321)
📝 Username: Natasha
🟢 Natasha joined the chat!
📊 Active users: 1
```

Asynchronous Client Stream

```
🟢 Natasha joined the chat!
🟢 Ali joined the chat!
Natasha: Hello everyone! Welcome to the channel.
Ali: Hi Natasha! The multi-threading is working perfectly.
```

Conclusions & Future System Roadmap

The project confirms that multi-threaded sock ve high performance networks. Next development milestones focus on securing these pipelines:

Encryption Mesh

Integrate RSA/AES asymmetric cryptographic structures for complete message privacy protection.

Database Layer

Embed structured SQLite or PostgreSQL backends to securely index chat archives.

Binary Files

Build custom chunk-based protocols to transfer binary media packages natively.

Advanced Auth

Implement secure salt-hashed credential models for multi-tenant verification.

Image Sources

https://png.pngtree.com/thumb_back/fw800/background/20251126/pngtree-abstract-digital-network-data-flow-with-glowing-teal-and-orange-connections-image_20424572.webp

Source: pngtree.com